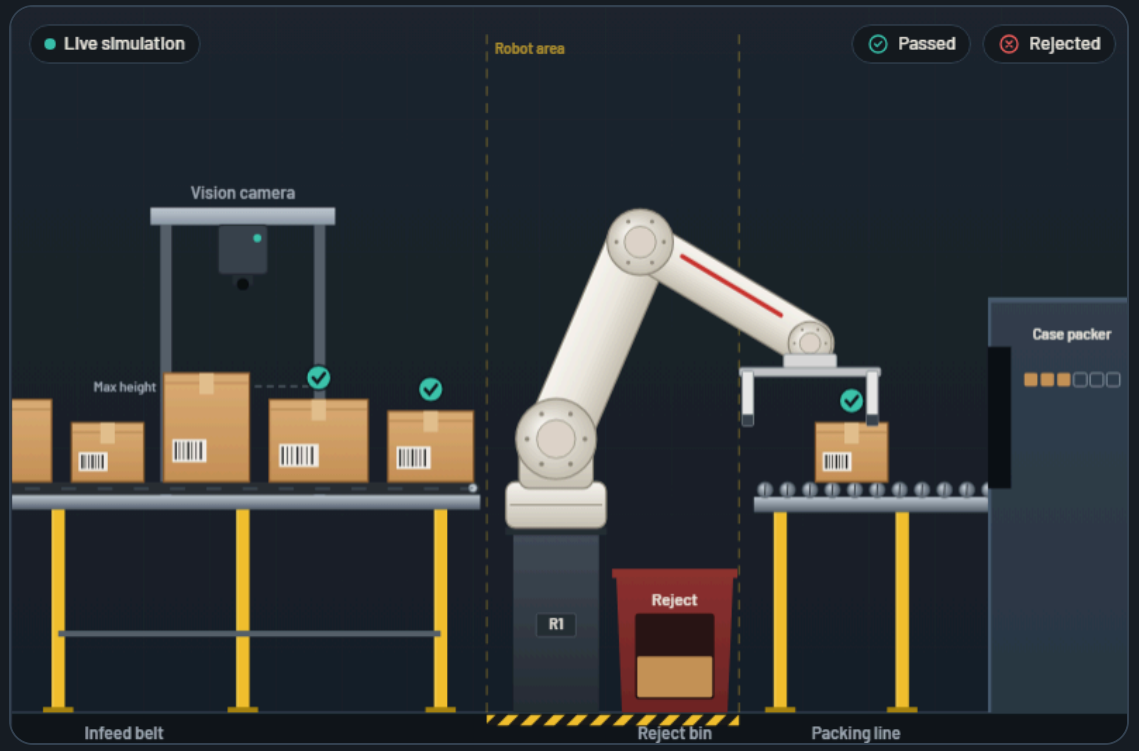
Robot Cell HMI

A simulated factory control screen, explained for beginners

Built with

- JavaScript
- HTML
- CSS
- C#
- SQL
- JSON

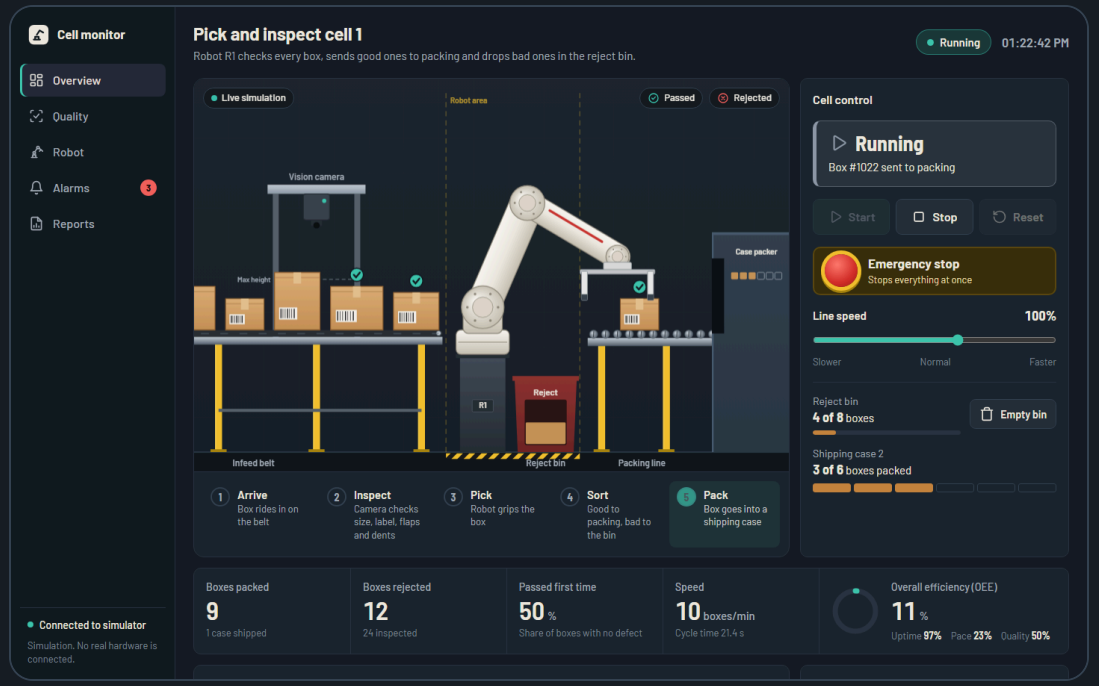


What is this project?

Think of it as a video game version of a small factory, with the real control screen a factory worker would use.

A robot checks boxes on a conveyor belt. Good boxes get packed, bad boxes get thrown out. You watch and control it from one screen.

- The robot, belts and camera move live, like the real thing
- You can start, stop and fix problems with buttons
- Everything is simulated, so no real machines are needed
- It keeps a history so you can review a whole shift



Words you will hear

Factories have their own vocabulary. Here is what each word means in plain language.

HMI

Human Machine Interface

The screen a person uses to watch and control a machine. This dashboard is an HMI.

SCADA

Supervisory Control and Data Acquisition

A system that watches many machines, collects their data and raises alarms.

PLC

Programmable Logic Controller

The small computer inside a machine that actually moves the motors. Our simulator pretends to be one.

Simulation

A program that behaves like a real machine, so you can learn and test without the hardware.

Historian

A program that records everything a machine does into a database, so you can look back later.

OEE

Overall Equipment Effectiveness

One score for how well the machine is doing. It combines uptime, speed and quality.

The factory we simulate

Every box goes through three steps, then gets sorted.

1

Box arrives

A box rides in on the conveyor belt.

2

Camera checks it

A vision camera looks for four kinds of defect.

- Open flap
- Missing label
- Dented corner
- Wrong size

3

Robot picks it up

The robot arm grips the box and lifts it off the belt.

Good box

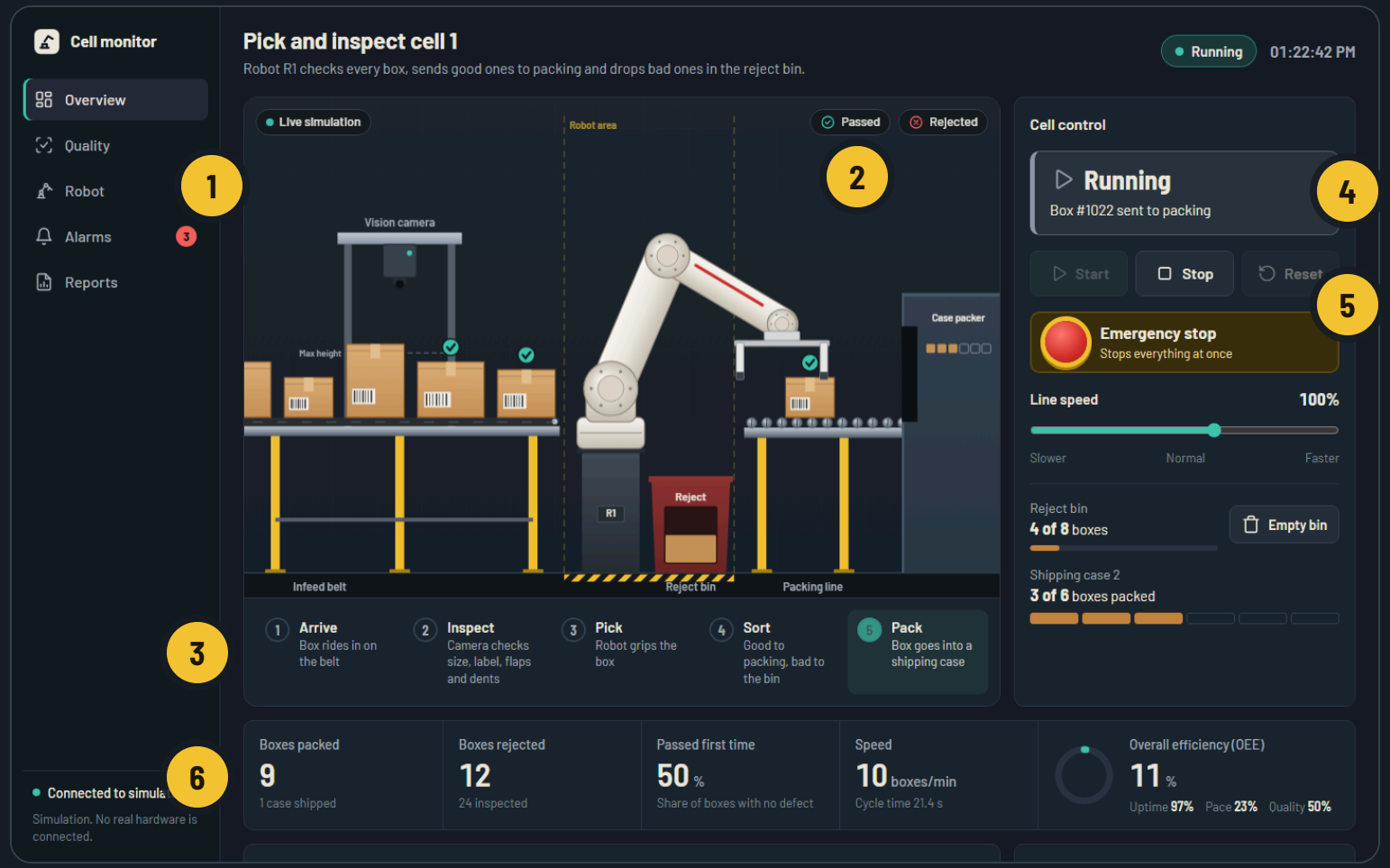
Placed on the packing line, then packed 6 to a shipping case.

Bad box

Dropped into the red reject bin. Someone must empty it when it fills up.

About 11 boxes a minute at normal speed. Roughly 1 in 10 boxes has a defect.

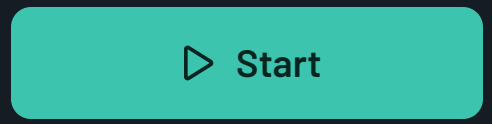
A tour of the main screen



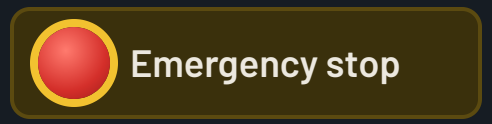
1. **Pages menu**
   Overview, Quality, Robot, Alarms and Reports
2. **Live animation**
   The robot, belts and camera, moving in real time
3. **Process steps**
   Lights up the step happening right now
4. **Status banner**
   Says what is happening and what you should do next
5. **Controls**
   Start, Stop, Reset, emergency stop and line speed
6. **Today's numbers**
   Boxes packed, rejected, pass rate, speed and OEE

Controlling the cell

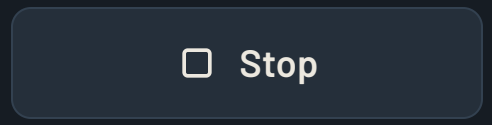
Six controls are all you need. Buttons you can't use right now are greyed out, so you can't press the wrong one.



Runs the cell. Boxes start arriving and the robot gets to work.



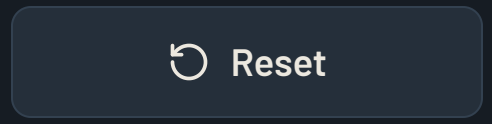
Stops everything at once. Use it if something looks dangerous.



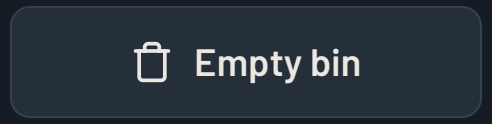
Stops politely. The robot finishes the box it is holding first.



Line speed. From 25% to 150%. Faster makes more boxes, but the robot heats up.



Clears a fault after you have fixed the problem. Then press Start again.



Empties the reject bin. When it is full, the robot waits for you.

Colours always mean the same thing

Most of the screen is calm grey. Bright colour only appears when something needs your attention.

Teal

Running normally, or a box passed inspection.

"Running", ✓ passed

Yellow

A warning. Not dangerous, but you should do something soon.

"Reject bin is full"

Red

An alarm. The cell has stopped and needs you now.

"Emergency stop", ✕ rejected

Grey

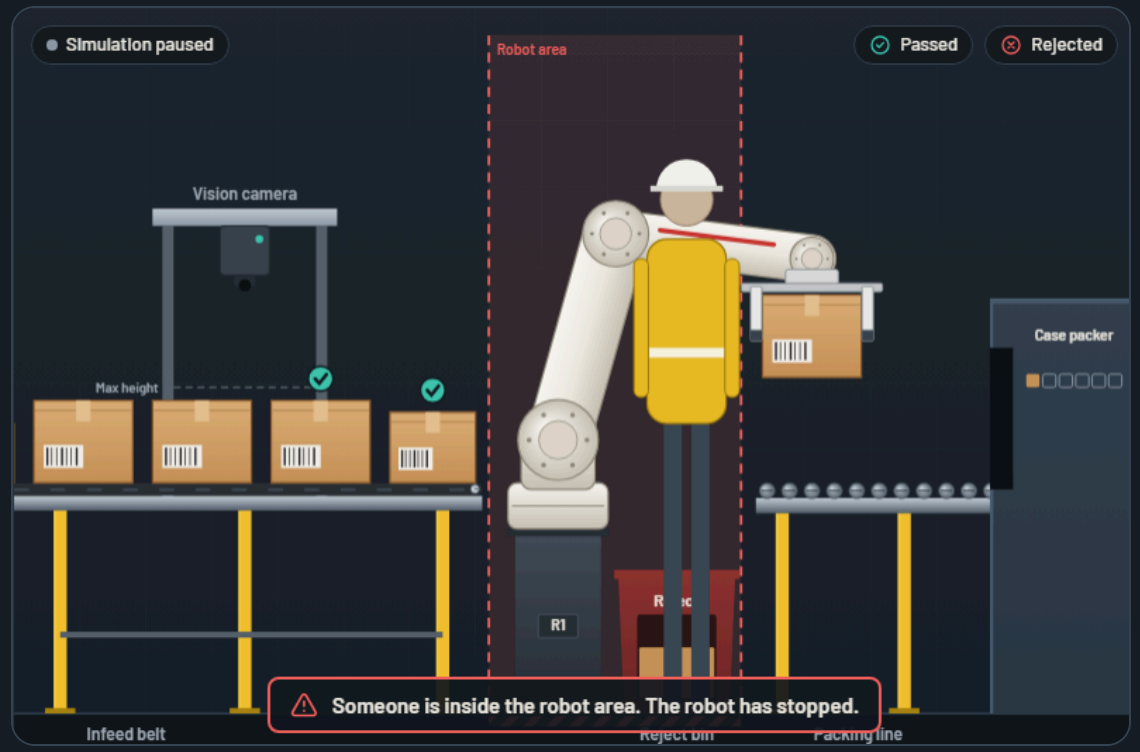
Stopped or waiting. Nothing is wrong.

"Stopped", "Ready"

Colour is never the only clue. Every state also has an icon and words, so colour-blind users can read it too.

Practise a problem

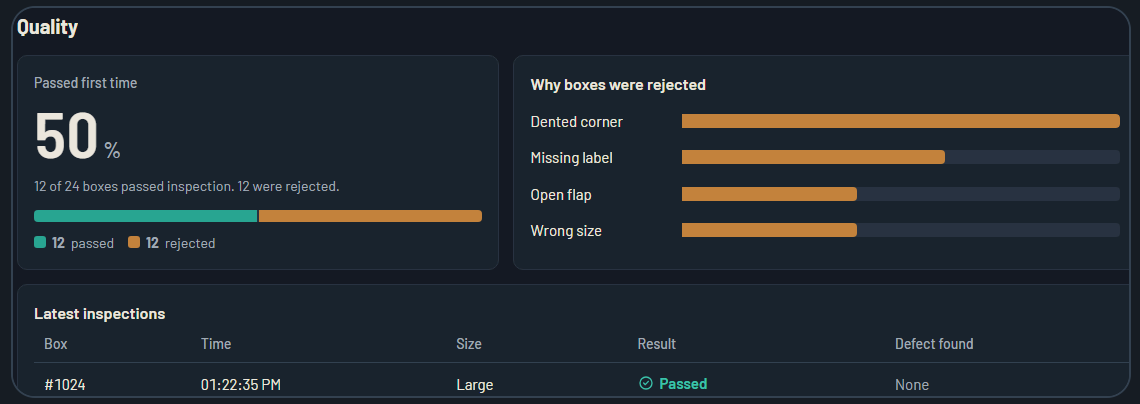
Four buttons cause a fault on purpose, so you can learn what to do before it happens for real.



"Walk into the robot area": the area turns red, the robot freezes and the banner says what to do.

Practice button	What happens	How to fix it
Walk into the robot area	Safety stop. Everything freezes.	Wait for the person to leave, press Reset, then Start
Jam the infeed belt	The belt stops with an alarm.	Press Reset, then Start
Fill the reject bin	The robot holds bad boxes and waits.	Press Empty bin
Send a bad batch	Half the boxes are faulty. A "High reject rate" warning appears.	Press the button again to stop it

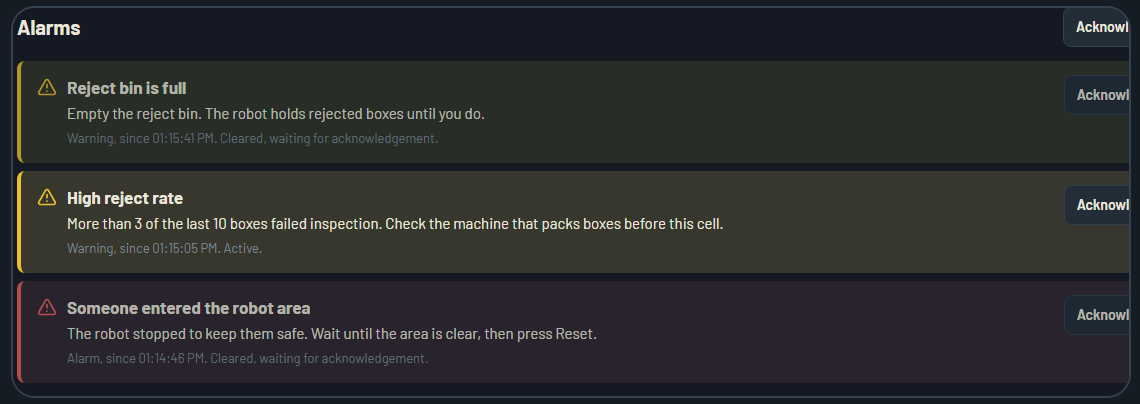
The other pages, from the menu on the left



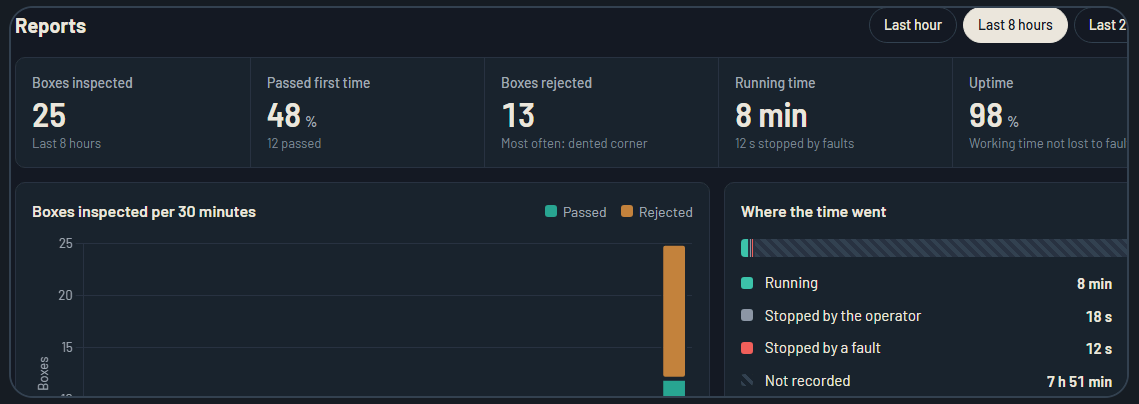
Quality Pass rate, reject reasons and the latest inspections



Robot What it is doing, arm angles, motor heat, cycle time

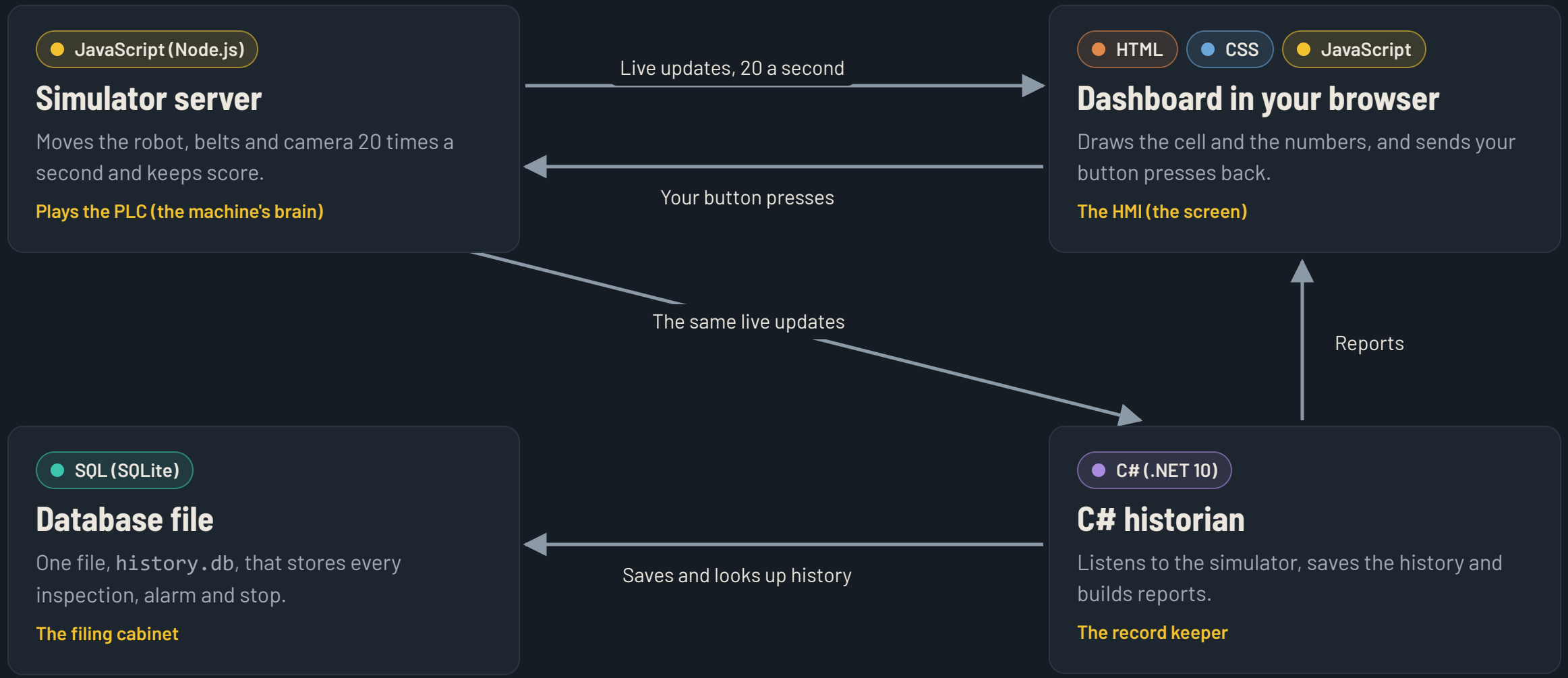


Alarms Each alarm with what to do, plus the event log



Reports Shift history from the C# historian, CSV and print

How the pieces fit together



Every message between the parts is written in JSON, a simple text format both JavaScript and C# understand.

The languages used, and what each one does

Six languages, each doing the job it is best at.

JavaScript

In `server.js`, `sim/cell.js`, `public/js/`

Runs the simulation and the web server, draws the moving robot and makes the buttons work.

HTML

In `public/index.html`

Builds the page itself: the menu, panels, buttons, tables and text.

CSS

In `public/css/style.css`

Makes it look right: colours, fonts, layout, the phone version and the print version.

C#

In `historian/*.cs`

The historian service. Records history in the background and serves the Reports page.

SQL

Inside `historian/HistoryStore.cs`

Saves data into the database and asks questions like "how many boxes failed today?"

JSON

In every message, and in `appsettings.json`

A simple text format the parts use to send data to each other.

Why these languages?

Each choice has a reason.

JavaScript	**It is the only language web browsers run.** With Node.js the server can use the same language, so you only need to learn one language for both. It is good at fast, live updates.
HTML	**It is the standard way to build any web page.** The dashboard opens on a PC, tablet or phone without installing anything.
CSS	**It keeps the look separate from the content.** One file changes the colours, the phone layout or the print layout without touching the page.
C#	**It is widely used in real industrial software**, such as SCADA, MES and Windows HMIs. It catches mistakes before the program runs, and it is fast and stable for services that run all day.
SQL	**It is the standard language for databases.** SQLite keeps the whole database in one file with no setup.
JSON	**Both JavaScript and C# read it easily**, and people can read it too. That makes it ideal for sending data between the parts.

Tools and libraries

Ready-made building blocks, so the project didn't have to start from zero.

Node.js	Runs JavaScript outside the browser, on the server
Express	A simple web server that sends the dashboard to your browser
ws (WebSocket)	Keeps a live two-way connection open for instant updates
npm and dotnet	Command-line tools that install and run each part
HTML canvas	The browser feature used to draw the animated robot
.NET 10 and ASP.NET Core	Runs the C# historian as a small web service
Microsoft.Data.Sqlite	Lets C# read and write the SQLite database
Chart.js	Draws the bar and line charts
Barlow font	An industrial-style typeface, free from Google Fonts
VS Code	The code editor used to write and run everything

What is in the project folder

Each file has one job.

Custom HMI-SCADA Project/	
server.js	Starts the simulator server (JavaScript)
sim/	
cell.js	The whole simulated factory: robot, belts, camera, alarms (JavaScript)
public/	Everything the browser shows
index.html	The page layout for all five pages (HTML)
css/style.css	Colours, layout, phone and print versions (CSS)
js/cellView.js	Draws the animated robot cell (JavaScript)
js/app.js	Fills in the dashboard and sends button presses (JavaScript)
historian/	The C# historian service
*.cs (5 files)	Records history and builds reports (C# with SQL inside)
README.md	Full instructions

How to run it

Open the project folder in VS Code and use its built-in terminal.

1. **Install the tools once**

   Node.js, plus the .NET 10 SDK if you want the Reports page.

2. **Download the libraries once**

   ```
   npm install
   ```

3. **Start the simulator**

   ```
   npm start
   ```

   Then open http://localhost:3000 and press Start.

4. **Optional: start the historian**

   ```
   npm run historian
   ```

   Run it in a second terminal. The Reports page fills in.

Just want a quick look?

Double-click `public/index.html`. The simulation then runs inside your browser, with no server needed.

The only things missing are the shared history and the Reports page, because those need the servers.

Made easy to use on purpose

Anyone should be able to run the cell without training.

Plain-language messages

The banner always says what is happening and what to do next, for example "Wait until the area is clear, then press Reset."

Big, clear buttons

Every button has an icon and a word. Buttons you can't use are greyed out.

One meaning per colour

Teal is fine, yellow is a warning, red is an alarm. Words always go with the colour.

Numbered process steps

You can see exactly which of the 5 steps the current box is in.

Safe practice mode

Trigger faults on purpose and learn how to recover, without risk.

Works anywhere

The layout adapts to a phone or tablet, and reports print on clean white paper.

Ideas for what to learn next

Each idea builds on what is already there.

Connect a real PLC

Swap the simulator for real hardware using Modbus or OPC UA. The dashboard stays the same.

Add a second robot

Extend the simulation with another station on the line.

Add operator logins

Only trained users may press Reset or change the speed.

Write automatic tests

Check the C# report maths with unit tests every time the code changes.

Read the README file in the project folder for every detail.